Discrete Structures

Title: Mini Logic Inference Engine in C++

Submitted To: Dr Ghousia Usman

Submitted By: Huda Zahid, Insha Rukhsar

Date: 28-11-2025

1.Problem Statement

Topic: AI Knowledge Base / Inference Engine
Subtopic: Predicate Logic & Inference Rules
Description: Build a mini rule-based AI that stores facts and rules (e.g., bird(x) → canFly(x)) and uses forward or backward chaining to infer new knowledge. This is useful for reasoning-based systems or expert systems.

The original task was to develop a **mini logic inference engine** capable of storing facts, rules, conjunctions, disjunctions, and performing logical inferences automatically. The engine should also provide explanations for each step of reasoning.

2. General Overview

A **logic inference engine** simulates **human logical reasoning**. It maintains a **knowledge base (KB)** with facts and rules, and applies logical inference rules to derive new knowledge. The engine implemented here supports:

- **Modus Ponens:** If P → Q and P is true, then Q is inferred.
- **Modus Tollens:** If P → Q and Q is false, then P is false.
- **Simplification:** From P AND Q, infer P or Q individually.
- **Disjunctive Syllogism:** From P OR Q, if P is false, then Q is true.
- **Resolution:** Combining disjunctions to derive new facts.

This engine is coded in **C++**, a high-level compiled language known for efficiency and low-level memory control. C++ is well-suited for implementing data structures like linked lists, which are used here to store rules, conjunctions, disjunctions, and resolution premises.

3. Major Breakdown of the Code

3.1 Helper Functions

- trim(): Removes whitespace from the start and end of a string.
- lower(): Converts a string to lowercase.
- normalize(): Combines trimming and lowercase conversion for consistent input handling.

3.2 Data Structures

- Lit: Represents a literal (atom or negated atom).
- RuleNode: Stores a rule of the form P → Q.
- ConjNode: Stores a conjunction P AND Q.
- DisjNode: Stores a disjunction P OR Q.
- ResNode: Stores resolution premises for advanced inference.

3.3 Knowledge Base Class (KB)

The **KB class** stores all facts, rules, conjunctions, disjunctions, and resolution premises.

❖ - Key functions:

- addRule(), addFactInput(), addConjInput(), addResInput(): Add knowledge.

- show(): Display current KB.

- getFactVal(), setFactVal(): Access or update fact truth values.

- inferOnce(): Perform a single inference step.

3.4 Inference Engine

❖ Applies classical logic rules:

1. Simplification
2. Modus Ponens
3. Modus Tollens
4. Disjunctive Syllogism
5. Hypothetical Syllogism
6. Conjunction
7. Resolution

❖ Provides explanations for each inference.

3.5 Main Program

❖ Menu-driven interface allows:

1. Adding rules (P → Q)
2. Adding facts (X, NOT X, A OR B)

3. Adding conjunctions (P AND Q)
4. Adding resolution premises ((P OR Q) AND (NOT P OR R))
5. Showing KB
6. Inferring one step
7. Inferring until no new conclusion
8. Clearing KB
9. Exiting program

4. Sample Inputs & Output

Real-world scenario (Weather & Road Conditions):

Type	Input
Rule	Rain -> WetRoad
Rule	Snow -> SlipperyRoad
Rule	WetRoad -> TrafficSlow
Fact	It is raining
Fact	Cold OR Sunny
Fact	It is snowing
Conjunction	Rain AND Cold
Resolution	(Rain OR Cold) AND (NOT Rain OR TrafficSlow)

Output:

1. Knowledge Base (KB) After Input

Step: After entering all facts, rules, conjunctions, disjunctions, and resolution premises, select **option 5 (Show KB)**.

Screenshot:

After inserting inputs initially:

```
=== Atomic Facts ===
rain = TRUE
snow = TRUE

=== Disjunctions (P OR Q) ===
cold OR sunny

=== Conjunctions (P AND Q) ===
<no conjunctions>

=== Rules (P -> Q) ===
wetroad -> trafficslow
snow -> slipperyroad
rain -> wetroad

=== Resolution Premises ===
rain or cold AND not rain or trafficslow
```

After running full inference:

```
=== Atomic Facts ===
rain = TRUE
wetroad = TRUE
snow = TRUE
slipperyroad = TRUE
trafficslow = TRUE

=== Disjunctions (P OR Q) ===
cold OR sunny

=== Conjunctions (P AND Q) ===
slipperyroad AND trafficslow
snow AND trafficslow
snow AND slipperyroad
wetroad AND trafficslow
wetroad AND slipperyroad
wetroad AND snow
rain AND trafficslow
rain AND slipperyroad
rain AND snow
rain AND wetroad

=== Rules (P -> Q) ===
rain -> trafficslow
wetroad -> trafficslow
snow -> slipperyroad
rain -> wetroad

=== Resolution Premises ===
rain or cold AND not rain or trafficslow
```

- **Explanation:** The first screenshot shows the current state of the knowledge base with all the facts, rules, conjunctions, disjunctions, and resolution premises entered. It is ready for the inference engine to process.
- The second screenshot shows the state of kb after full inference, the system has inferred new facts based on its own knowledge.

2. Inference Step (Single Step)

Step: Select **option 6 (Infer one step)**.

Screenshot:

```
=== Inference AI (No STL) ===
1) Add rule (P -> Q)
2) Add fact (X, not X, A or B)
3) Add conjunction premise (P AND Q)
4) Add resolution premise ((P OR Q) AND (NOT P OR R))
5) Show KB
6) Infer one step
7) Infer until no new conclusion
8) Clear KB
0) Exit
Choose: 6

--------------------------------------------------
? Rule Used: Modus Tollens
? Given Facts
If rain then wetroad
wetroad is FALSE.

? Conclusion
Therefore NOT rain

Structure:
If P -> Q
¾Q
Therefore ¾P
--------------------------------------------------
```

Explanation: The screenshot above shows the inference engine applying a single logical rule to the knowledge base. It highlights which facts were used, which rule was applied, and what conclusion was drawn in this single step of reasoning.

3. Full Inference (Until No New Conclusion)

Step: Select **option 7 (Infer until no new conclusion)**.

Output:

--- Starting Full Inference ---

? Rule Used: Modus Ponens

? Given Facts

If snow then slipperyroad

snow is TRUE.

? Conclusion

Therefore slipperyroad

Structure:

If P -> Q

P

Therefore Q

? Rule Used: Modus Ponens

? Given Facts

If rain then wetroad

rain is TRUE.

? Conclusion

Therefore wetroad

Structure:

If P -> Q

P

Therefore Q

? Rule Used: Modus Ponens

? Given Facts

If wetroad then trafficslow

wetroad is TRUE.

? Conclusion

Therefore trafficslow

Structure:

If P -> Q

P

Therefore Q

? Rule Used: Hypothetical Syllogism

? Given Facts

rain -> wetroad

wetroad -> trafficslow

? Conclusion

Therefore rain -> trafficslow

Structure:

P -> Q

Q -> R

Therefore P -> R

? Rule Used: Conjunction

? Given Facts

rain is TRUE.

wetroad is TRUE.

? Conclusion

Therefore rain AND wetroad

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

rain is TRUE.

snow is TRUE.

? Conclusion

Therefore rain AND snow

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

rain is TRUE.

slipperyroad is TRUE.

? Conclusion

Therefore rain AND slipperyroad

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

rain is TRUE.

trafficslow is TRUE.

? Conclusion

Therefore rain AND trafficslow

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

wetroad is TRUE.

snow is TRUE.

? Conclusion

Therefore wetroad AND snow

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

wetroad is TRUE.

slipperyroad is TRUE.

? Conclusion

Therefore wetroad AND slipperyroad

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

wetroad is TRUE.

trafficslow is TRUE.

? Conclusion

Therefore wetroad AND trafficslow

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

snow is TRUE.

slipperyroad is TRUE.

? Conclusion

Therefore snow AND slipperyroad

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

snow is TRUE.

trafficslow is TRUE.

? Conclusion

Therefore snow AND trafficslow

Structure:

P

Q

Therefore P ^ Q

? Rule Used: Conjunction

? Given Facts

slipperyroad is TRUE.

trafficslow is TRUE.

? Conclusion

Therefore slipperyroad AND trafficslow

Structure:

P

Q

Therefore P ^ Q

--- Inference Complete ---

Explanation: The output above shows the complete inference process where the engine iteratively applies all logical rules until no further conclusions can be drawn. All intermediate steps, including the rules applied and new facts inferred, are visible in the output, demonstrating the reasoning process of the engine.

5. Program Capabilities

This mini logic inference engine can:

- - Store facts
- - Store rules (P → Q)
- - Store conjunctions (P AND Q)
- - Store disjunctions (P OR Q)
- - Perform inference using:
  - - Modus Ponens $(p \wedge (p \rightarrow q)) \rightarrow q$
  - - Modus Tollens $(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$
  - - Simplification $(p \wedge q) \rightarrow p$
  - - Disjunctive Syllogism $((p \vee q) \wedge \neg p) \rightarrow q$
  - - Conjunction $((p) \wedge (q)) \rightarrow (p \wedge q)$
  - - Hypothetical Syllogism $((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$
  - - Resolution $((p \vee q) \wedge (\neg p \vee r)) \rightarrow (q \vee r)$
- - Provide **detailed step-by-step explanations**.

6. Conclusion

The mini logic inference engine demonstrates **classical logical reasoning** using a C++ implementation. Users can input facts and rules, then watch as the engine automatically infers new knowledge. It is a foundational example for reasoning-based AI or expert systems.